

Trabajo Final de Inteligencia Artificial

Tema 2

Construcción de resúmenes como selección óptima de segmentos

Autor: Rubén Martínez Rojas

Grupo C-311

Curso 2025–2026

12 de junio de 2026

Índice

1. Introducción

La generación automática de resúmenes constituye una de las aplicaciones más relevantes del procesamiento del lenguaje natural, especialmente en contextos donde es necesario condensar grandes volúmenes de información para facilitar su consulta y comprensión. En el caso de los contenidos audiovisuales educativos, la duración de los materiales y la cantidad de información presentada dificultan la revisión rápida de los temas abordados, lo que hace necesario disponer de mecanismos capaces de producir versiones resumidas que preserven las ideas principales.

El presente trabajo desarrolla una solución para el Tema 2 del Proyecto Final de la asignatura Inteligencia Artificial, titulado “Construcción de resúmenes como selección óptima de segmentos”. El problema consiste en seleccionar y ordenar un subconjunto de fragmentos extraídos de un video educativo de manera que el resumen resultante represente adecuadamente el contenido original sin exceder una duración máxima establecida. La calidad del resumen depende tanto de la importancia individual de los fragmentos seleccionados como de la coherencia existente entre ellos cuando se presentan de forma consecutiva.

Desde el punto de vista de la optimización, el problema puede interpretarse como una tarea de selección bajo restricciones. Cada fragmento posee una duración asociada que consume parte del presupuesto temporal disponible, mientras que su incorporación al resumen aporta una determinada utilidad relacionada con su relevancia informativa. Adicionalmente, el orden en que los fragmentos son presentados influye en la calidad global del resumen, por lo que la solución debe considerar simultáneamente la selección de fragmentos y su secuencia de reproducción.

Como parte de los requisitos del proyecto, se incorpora un modelo de lenguaje de gran tamaño (LLM) dentro del sistema. En esta propuesta, el modelo de lenguaje se utiliza para evaluar la relevancia de cada fragmento y la coherencia entre pares de fragmentos. Estas valoraciones son posteriormente empleadas por el algoritmo de optimización para construir candidatos de resumen. De esta forma, el LLM aporta conocimiento semántico sobre el contenido textual mientras que el proceso de búsqueda y optimización permanece a cargo de un algoritmo combinatorio independiente.

La solución desarrollada se basa en una arquitectura compuesta por tres etapas principales. En primer lugar, se realiza un precálculo de puntuaciones de relevancia y coherencia mediante el modelo de lenguaje. Posteriormente, un algoritmo de beam search explora distintas combinaciones de fragmentos respetando la restricción de duración y conservando únicamente las alternativas más prometedoras. Finalmente, los mejores candidatos obtenidos son sometidos a una evaluación global para seleccionar la versión final del resumen.

El objetivo del trabajo es diseñar e implementar un sistema capaz de generar resúmenes coherentes y representativos a partir de fragmentos de videos educativos, integrando técnicas de optimización y modelos de lenguaje dentro de un mismo proceso. Asimismo, se estudia el comportamiento de la solución mediante diferentes instancias de prueba y configuraciones experimentales con el fin de analizar sus fortalezas, limitaciones y posibilidades de mejora.

2. Descripción del problema

2.1. Contexto

Los videos educativos constituyen una fuente importante de información para el aprendizaje en diferentes áreas del conocimiento. Sin embargo, la duración de estos materiales puede dificultar la localización rápida de conceptos específicos o la revisión general de los contenidos tratados. Una alternativa para reducir este problema consiste en generar resúmenes que concentren las ideas más importantes del video y permitan al usuario obtener una visión general de su contenido en un tiempo significativamente menor.

En este trabajo se considera un escenario en el que cada video ha sido previamente segmentado en fragmentos temporales y cada fragmento dispone de una transcripción textual asociada. A partir de estos fragmentos, el sistema debe construir un resumen que represente adecuadamente la información contenida en el material original. Debido a que el resumen debe respetar una duración máxima establecida, no es posible incluir todos los fragmentos disponibles, por lo que resulta necesario decidir cuáles conservar y cuáles descartar.

La generación del resumen no depende únicamente de la importancia individual de cada fragmento. Un resumen compuesto por fragmentos muy relevantes puede resultar difícil de comprender si las transiciones entre ellos son poco naturales o si el orden de presentación rompe la continuidad temática. Por esta razón, además de valorar la información aportada por cada fragmento, es necesario considerar la coherencia existente entre los elementos seleccionados.

2.2. Definición del problema

Sea un conjunto de fragmentos extraídos de un video educativo. Cada fragmento está caracterizado por una transcripción textual, una duración y la información temporal correspondiente a su posición dentro del video original. Además, se dispone de un límite de duración que establece la longitud máxima permitida para el resumen final.

El problema consiste en seleccionar un subconjunto de fragmentos cuya duración total no exceda dicho límite y determinar un orden de reproducción para los fragmentos seleccionados. El objetivo es obtener un resumen que preserve la mayor cantidad posible de información relevante y que mantenga una secuencia coherente entre sus diferentes partes.

Para evaluar la calidad de una solución se consideran dos criterios principales. El primero es la relevancia de los fragmentos individuales, entendida como la contribución informativa de cada uno al contenido global del video. El segundo es la coherencia entre fragmentos consecutivos, entendida como el grado en que dos fragmentos pueden aparecer uno después del otro manteniendo continuidad temática y discursiva. Una solución de alta calidad debe equilibrar ambos criterios mientras satisface la restricción de duración impuesta.

La naturaleza combinatoria del problema surge del elevado número de posibles subconjuntos y ordenaciones que pueden construirse a partir de los fragmentos disponibles. A medida que aumenta la cantidad de fragmentos, el espacio de búsqueda

crece rápidamente, lo que dificulta encontrar soluciones de calidad mediante una exploración exhaustiva. Por esta razón resulta necesario emplear técnicas de búsqueda y optimización que permitan explorar únicamente las alternativas más prometedoras.

2.3. Objetivos

El objetivo general del trabajo es desarrollar un sistema capaz de construir resúmenes de videos educativos mediante la selección y ordenación de fragmentos bajo una restricción de duración máxima, incorporando un modelo de lenguaje como parte funcional del proceso de evaluación.

Para alcanzar este objetivo se propone representar formalmente el problema como una tarea de optimización en la que cada solución corresponde a una secuencia ordenada de fragmentos. Asimismo, se busca utilizar un modelo de lenguaje para estimar la relevancia de los fragmentos y la coherencia entre pares de fragmentos, aprovechando su capacidad para analizar información textual y capturar relaciones semánticas que resultan difíciles de modelar mediante reglas manuales.

Además, se pretende diseñar e implementar un algoritmo de búsqueda capaz de generar resúmenes de calidad sin necesidad de explorar exhaustivamente todas las combinaciones posibles. Finalmente, se realizará una evaluación experimental sobre diferentes instancias de prueba con el propósito de analizar el comportamiento del sistema, estudiar la influencia del modelo de lenguaje en el proceso de selección y determinar las principales limitaciones de la propuesta desarrollada.

3. Modelado formal

3.1. Representación de fragmentos

La entrada del problema está compuesta por un conjunto finito de fragmentos obtenidos a partir de la transcripción de un video educativo. Cada fragmento contiene el texto correspondiente a una porción del video, así como información temporal asociada a su posición dentro del material original.

Formalmente, cada fragmento puede representarse mediante una tupla:

$$f_i = (t_i, d_i, s_i, e_i)$$

donde t_i representa el texto del fragmento, d_i su duración en segundos, s_i el instante de inicio y e_i el instante de finalización dentro del video original. Sea

$$F = \{f_1, f_2, \dots, f_n\}$$

el conjunto de todos los fragmentos disponibles. Además, se dispone de una duración máxima permitida para el resumen, denotada por D_{max} .

Una solución del problema consiste en construir una secuencia ordenada de fragmentos seleccionados:

$$S = (f_{i_1}, f_{i_2}, \dots, f_{i_k})$$

donde cada fragmento puede aparecer como máximo una vez.

3.2. Relevancia

Con el objetivo de estimar la importancia informativa de cada fragmento, se asocia una puntuación de relevancia a cada elemento del conjunto de entrada.

Sea

$$r_i \in [0, 1]$$

la relevancia asignada al fragmento (f_i) . Valores cercanos a 1 indican que el fragmento contiene información especialmente útil para la construcción de un resumen educativo, mientras que valores cercanos a 0 representan fragmentos con escasa contribución al contenido principal.

Estas puntuaciones son obtenidas mediante un modelo de lenguaje, el cual analiza el texto de cada fragmento de manera independiente y devuelve un valor numérico normalizado entre 0 y 1.

3.3. Coherencia

Además de la relevancia individual de los fragmentos, es necesario considerar la relación existente entre fragmentos consecutivos dentro del resumen.

Para cada par ordenado de fragmentos $((f_i, f_j))$ se define una puntuación de coherencia:

$$c_{ij} \in [0, 1]$$

que representa el grado de continuidad temática y discursiva existente al colocar (f_j) inmediatamente después de (f_i) .

Valores elevados de (c_{ij}) indican que ambos fragmentos forman una transición natural dentro del resumen, mientras que valores bajos sugieren cambios bruscos de tema o secuencias poco comprensibles para el lector.

Al igual que ocurre con la relevancia, estas puntuaciones son calculadas mediante un modelo de lenguaje a partir del contenido textual de ambos fragmentos.

3.4. Restricción de duración

La duración total del resumen no puede exceder el límite establecido para la instancia.

Si la solución seleccionada contiene los fragmentos

$$S = (f_{i_1}, f_{i_2}, \dots, f_{i_k}),$$

entonces debe cumplirse que

$$\sum_{j=1}^k d_{i_j} \leq D_{max}.$$

Esta restricción garantiza que el resumen generado respete la longitud máxima especificada para el problema.

3.5. Función objetivo

El objetivo consiste en encontrar una secuencia de fragmentos que maximice simultáneamente la relevancia del contenido seleccionado y la coherencia entre fragmentos consecutivos.

Para una solución

$$S = (f_{i_1}, f_{i_2}, \dots, f_{i_k}),$$

la calidad puede expresarse mediante una combinación de ambos criterios:

$$\text{Score}(S) = \sum_{j=1}^k r_{i_j} + \lambda \sum_{j=1}^{k-1} c_{i_j, i_{j+1}} \quad (1)$$

donde (λ) representa el peso asignado a la coherencia dentro de la evaluación total.

En la implementación desarrollada, la búsqueda utiliza un valor de $(\lambda = 0.25)$, lo que permite priorizar la selección de fragmentos relevantes sin ignorar la calidad de las transiciones entre ellos.

Por tanto, el problema consiste en encontrar la secuencia válida de fragmentos que maximice la función objetivo anterior y satisfaga simultáneamente la restricción de duración establecida para el resumen.

4. Diseño de la solución

4.1. Arquitectura general

La solución desarrollada sigue una arquitectura modular compuesta por tres etapas principales: evaluación semántica mediante un modelo de lenguaje, búsqueda combinatoria de candidatos y evaluación global de los resúmenes generados. Esta separación permite aprovechar las capacidades de comprensión textual de los modelos de lenguaje sin delegar en ellos el proceso de optimización, que permanece completamente controlado por algoritmos deterministas implementados en el sistema.

El flujo de ejecución comienza con la carga de una instancia del problema. Cada instancia contiene un conjunto de fragmentos obtenidos a partir de la transcripción de un video educativo y una duración máxima permitida para el resumen final. Una vez cargada la información, se inicia una fase de precálculo en la que se evalúan todos los fragmentos disponibles mediante el modelo de lenguaje.

Durante esta etapa se obtiene una puntuación de relevancia para cada fragmento y una matriz de coherencia entre todos los pares posibles de fragmentos. La relevancia representa la utilidad informativa de un fragmento dentro de un resumen educativo, mientras que la coherencia mide la calidad de la transición producida al situar un fragmento inmediatamente después de otro. Estas puntuaciones son

almacenadas en una estructura de datos que posteriormente será utilizada por el algoritmo de búsqueda.

Una vez completado el precálculo, el sistema ejecuta un algoritmo de beam search encargado de construir soluciones candidatas. A diferencia de otros enfoques en los que el modelo de lenguaje participa directamente durante la exploración, en esta propuesta todas las consultas al LLM se realizan antes de comenzar la búsqueda. De esta forma, el algoritmo puede explorar múltiples combinaciones de fragmentos utilizando únicamente las puntuaciones previamente calculadas, reduciendo significativamente el número de llamadas a la API y mejorando la eficiencia del proceso.

El beam search genera secuencias ordenadas de fragmentos que respetan la restricción de duración máxima y asigna una puntuación local a cada candidato en función de la relevancia acumulada y de la coherencia entre fragmentos consecutivos. Durante la exploración se conservan únicamente las alternativas más prometedoras, limitando así el crecimiento del espacio de búsqueda.

En lugar de seleccionar directamente el candidato con mejor puntuación local, el sistema conserva los mejores resultados obtenidos por el beam search y realiza una etapa adicional de refinamiento. En esta fase, el texto completo de cada resumen candidato es evaluado nuevamente mediante el modelo de lenguaje, que proporciona una valoración global basada en la relevancia del contenido seleccionado y en la coherencia de la secuencia resultante.

Finalmente, el resumen con mayor evaluación global es devuelto como solución del problema. El resultado consiste en una secuencia ordenada de fragmentos cuya duración total no supera el límite establecido y que busca maximizar simultáneamente la representatividad del contenido y la coherencia narrativa del resumen generado.

La Figura X resume el flujo general de funcionamiento del sistema.

Entrada de fragmentos → Evaluación mediante LLM → Beam Search →
Evaluación global del resumen → Resumen final

4.2. Preparación del dataset

El sistema desarrollado trabaja sobre instancias construidas a partir de transcripciones de videos educativos. Con el fin de disponer de datos adecuados para la experimentación, se implementó un proceso de preparación que transforma los archivos de subtítulos originales en instancias estructuradas del problema de optimización.

La generación de las instancias reales se realiza mediante el script `prepare_dataset.py`. Como entrada, el programa recibe archivos de subtítulos en formato SRT correspondientes a diferentes videos educativos. Cada bloque de subtítulos contiene un intervalo temporal y el texto pronunciado durante dicho intervalo. El sistema procesa estos archivos eliminando marcas no informativas y convirtiendo los tiempos a segundos para facilitar su tratamiento posterior.

Una vez cargados los subtítulos, los bloques individuales son agrupados en fragmentos de mayor tamaño mediante una estrategia de agregación temporal. El objetivo es obtener segmentos suficientemente largos para contener información con significado semántico completo, evitando al mismo tiempo fragmentos excesivamente extensos. En la implementación utilizada, los subtítulos consecutivos se agrupan

hasta alcanzar aproximadamente entre veinte y veinticinco segundos de duración acumulada.

Cada fragmento generado se representa mediante una estructura que almacena el texto concatenado, la duración del segmento y su posición temporal dentro del video original. Formalmente, cada instancia contiene una colección ordenada de fragmentos junto con la duración máxima permitida para el resumen.

Para determinar dicha restricción se calcula primero la duración total del material de entrada:

$$D_{total} = \sum_{i=1}^n d_i,$$

donde d_i representa la duración del fragmento f_i . A continuación, la duración máxima del resumen se fija como un veinticinco por ciento de la duración total disponible:

$$D_{max} = 0,25 \cdot D_{total}.$$

Esta elección permite generar resúmenes significativamente más breves que el contenido original manteniendo un margen suficiente para incluir los fragmentos más representativos.

Las instancias resultantes se almacenan en formato JSON dentro del directorio `data/instances/`. Cada archivo contiene la lista de fragmentos generados y la restricción temporal asociada. Los videos reales procesados producen instancias denominadas `video_1.json`, `video_2.json`, `video_3.json` y `video_4.json`, con tamaños comprendidos aproximadamente entre veinticinco y cincuenta fragmentos.

Además de las instancias completas, se generaron conjuntos derivados destinados a facilitar la experimentación. Por una parte, el script `prepare_mini_instances.py` permite construir versiones reducidas de los videos conservando únicamente los primeros fragmentos consecutivos. Estas instancias, denominadas `mini_video_*`, mantienen la estructura del problema original pero reducen considerablemente el coste computacional asociado a la evaluación mediante modelos de lenguaje.

Por otra parte, se construyeron instancias de escalado basadas en diferentes cantidades de contenido procedente de un mismo video. Estas instancias, identificadas como `dur_5min`, `dur_10min`, `dur_15min` y `dur_20min`, permiten analizar cómo evoluciona el comportamiento del sistema a medida que aumenta el número de fragmentos disponibles y, en consecuencia, el tamaño del espacio de búsqueda.

Finalmente, para complementar los datos reales, se diseñaron varias instancias sintéticas de pequeño tamaño. Estas instancias contienen únicamente cinco fragmentos y fueron construidas manualmente con el objetivo de reproducir situaciones específicas de interés experimental, como la presencia de fragmentos irrelevantes o la existencia de diferentes alternativas de ordenación. Su reducido tamaño permite comparar de forma controlada el comportamiento de los distintos solvers y facilita la interpretación de los resultados obtenidos.

La combinación de instancias reales, instancias derivadas e instancias sintéticas proporciona un conjunto de pruebas suficientemente variado para estudiar tanto la calidad de los resúmenes generados como las limitaciones computacionales de la propuesta desarrollada.

4.3. Evaluación mediante LLM

El modelo de lenguaje constituye el componente encargado de proporcionar las valoraciones semánticas utilizadas posteriormente por el algoritmo de optimización. Su función no consiste en generar directamente el resumen, sino en estimar la calidad informativa de los fragmentos y la coherencia existente entre ellos.

La evaluación se realiza en una fase de precálculo previa a la búsqueda combinatoria. De esta forma, todas las llamadas al modelo de lenguaje se ejecutan una única vez para cada instancia, evitando consultas repetidas durante la exploración del espacio de soluciones.

El sistema calcula dos tipos de puntuaciones.

En primer lugar, para cada fragmento individual se obtiene una puntuación de relevancia. Para ello se construye un prompt que solicita al modelo valorar la utilidad del fragmento dentro de un resumen educativo y devolver exclusivamente un número comprendido entre 0 y 1. Cuanto mayor sea la puntuación obtenida, mayor será la importancia atribuida al fragmento dentro del contenido global del video.

En segundo lugar, se calcula una puntuación de coherencia para cada par ordenado de fragmentos. En este caso el modelo recibe los textos de ambos fragmentos y debe estimar el grado de continuidad temática y discursiva que existe al colocar el segundo inmediatamente después del primero. Al igual que en la evaluación de relevancia, la respuesta esperada es un valor numérico normalizado entre 0 y 1.

Las puntuaciones obtenidas se almacenan en dos estructuras de datos. La primera es un vector de relevancias:

$$[R=(r_1, r_2, \dots, r_n),]$$

donde (r_i) *representa la relevancia del fragmento* (f_i) .

La segunda es una matriz de coherencia:

$$[C= \begin{bmatrix} 0 & c_{12} & \cdots & c_{1n} & c_{21} & 0 & \cdots & c_{2n} & \vdots & \vdots & \ddots & \vdots & c_{n1} \\ c_{n2} & \cdots & 0 & & & & & & & & & & \end{bmatrix},]$$

donde cada elemento (c_{ij}) *expresa la coherencia obtenida al situar el fragmento* (f_j) *después del fragmento* (f_i) .

Con el objetivo de reducir el número de llamadas a la API y acelerar los experimentos, el sistema incorpora un mecanismo de caché persistente. Cada prompt enviado al modelo se almacena junto con su respuesta, de manera que futuras ejecuciones pueden reutilizar automáticamente los resultados ya calculados.

La implementación permite utilizar diferentes proveedores de modelos de lenguaje mediante una interfaz común. Actualmente se soportan Google Gemini y APIs compatibles con OpenAI, incluyendo Groq. Esta abstracción facilita la comparación entre modelos sin necesidad de modificar el resto del sistema.

El resultado final de esta etapa es una matriz completa de puntuaciones de relevancia y coherencia que será utilizada posteriormente por el algoritmo de beam search para construir los resúmenes candidatos.

4.4. Beam Search

Una vez calculadas las puntuaciones de relevancia y coherencia mediante el modelo de lenguaje, el sistema debe determinar qué fragmentos incluir en el resumen final. Dado que el número de posibles combinaciones crece rápidamente con la can-

tividad de fragmentos disponibles, una búsqueda exhaustiva resulta poco práctica incluso para instancias de tamaño moderado. Por esta razón se emplea un algoritmo de *beam search*, capaz de explorar únicamente las alternativas más prometedoras del espacio de búsqueda.

El objetivo del algoritmo es construir progresivamente secuencias ordenadas de fragmentos que respeten la restricción de duración máxima y maximicen la función objetivo definida en la Sección 3. Cada estado de búsqueda representa una solución parcial formada por una secuencia de fragmentos ya seleccionados, la puntuación acumulada obtenida hasta el momento y la duración total consumida.

Inicialmente el algoritmo parte de una solución vacía. En cada iteración se generan nuevas alternativas añadiendo un fragmento no utilizado a cada una de las secuencias parciales presentes en el beam. Una expansión únicamente se considera válida cuando la duración total resultante no supera el límite establecido para la instancia.

La puntuación de una solución parcial se calcula de forma incremental. Cuando se añade un nuevo fragmento (f_j), *la puntuación aumenta en función de su relevancia individual, si existe un*

$$[\Delta = r_j + \lambda, c_{ij},]$$

donde (f_i) *representa el último fragmento de la secuencia actual y (λ) es el peso asignado a la coherencia.*

Tras generar todas las expansiones posibles, los candidatos se ordenan según su puntuación acumulada y únicamente se conservan los mejores. Este conjunto reducido constituye el nuevo beam que será expandido en la siguiente iteración. El parámetro que controla el número máximo de candidatos conservados recibe el nombre de *beam width*.

La principal ventaja de este enfoque es que permite explorar múltiples alternativas simultáneamente sin necesidad de mantener todas las combinaciones posibles. Mientras que una búsqueda exhaustiva tendría un crecimiento exponencial, el beam search limita deliberadamente el número de estados activos, logrando tiempos de ejecución mucho más reducidos a costa de renunciar a la garantía de optimalidad global.

Además de devolver el mejor candidato encontrado, la implementación desarrollada permite conservar varios resúmenes de alta calidad. Para ello se mantiene una lista de los (M) candidatos con mayor puntuación local obtenidos durante la exploración. Esta característica resulta especialmente útil porque posibilita una fase posterior de refinamiento mediante un juez global basado en LLM.

En la configuración utilizada durante los experimentos se emplea un ancho de beam de diez candidatos (*beam width* = 10). Este valor proporciona un equilibrio adecuado entre calidad de las soluciones y coste computacional, permitiendo explorar diferentes alternativas sin que el tiempo de ejecución aumente de forma significativa.

El resultado de esta etapa es un conjunto reducido de resúmenes candidatos que cumplen la restricción de duración y presentan una combinación favorable de relevancia y coherencia. Estos candidatos son posteriormente evaluados mediante un análisis global para seleccionar la versión final del resumen.

4.5. Refinamiento mediante juez macro

5. Implementación

5.1. Tecnologías utilizadas

5.2. Estructura del proyecto

5.3. Configuración del LLM

5.4. Caché de respuestas

6. Metodología experimental

6.1. Dataset utilizado

6.2. Instancias de prueba

6.3. Configuración experimental

6.4. Métricas

7. Resultados y análisis

7.1. Casos sintéticos

7.2. Casos benchmark

7.3. Escalabilidad

7.4. Discusión

8. Limitaciones y mejoras futuras

9. Conclusiones

A. Configuraciones experimentales

B. Prompts utilizados

C. Resultados completos